Movies classification

By Alicja Jakowczuk, Tomasz Białecki

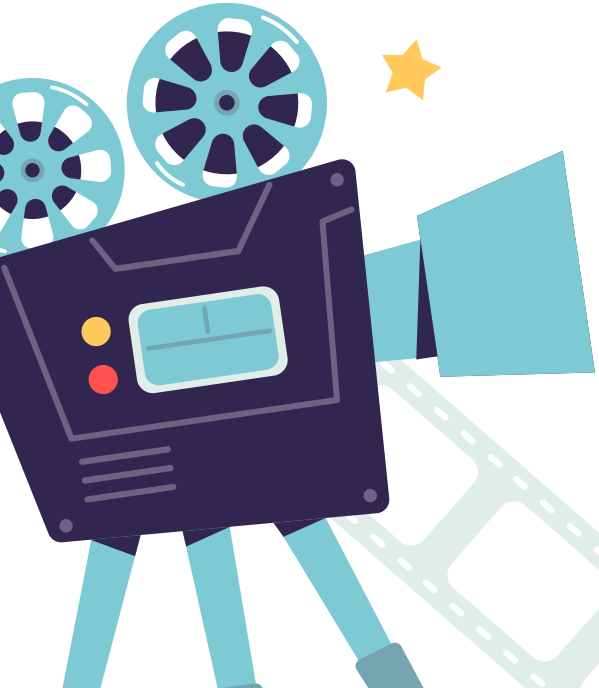


Table of contents

1. Introduction

In this project, we used real-world data from Letterboxd to compare how effectively different machine learning models can predict a film's quality rating using features like genre, director, and popularity metrics.

1. Data before preprocessing

Attribute	Data Type
Film_title	str
Director	str
Genres	str
Original_language	str
Studios	str
Description	str
Average_rating	float64
Runtime	float64

Attribute	Data Type
Watches	int64
List_appearances	int64
Likes	int64
Fans	int64
Total_ratings	int64
Lowest★	int64
Medium★★★	int64
Highest★★★★★	int64

1. Data before preprocessing- example

Attribute	Data Type
Film_title	Lost in Translation
Director	Sofia Coppola
Genres	['Drama', 'Comedy', 'Romance']
Original_language	English
Studios	['American Zoetrope', ...]
Description	Two lost souls visiting Tokyo ...
Average_rating	3.79
Runtime	102.0

Attribute	Data Type
Watches	1,596,190
List_appearances	254,180
Likes	493,248
Fans	38,000
Total_ratings	1,076,949
Lowest★	15,167
Medium★★★	193,717
Highest★★★★★	1,076,949

1. Data after preprocessing

Attribute	Data Type
Fi tle	str
Director	str
Genres	str
Original_language	str
Studios	str
De ription	str
Average_rating	float64
Runtime	float64

One-hot encoding

One-hot encoding

One-hot encoding

Used for creating classification column

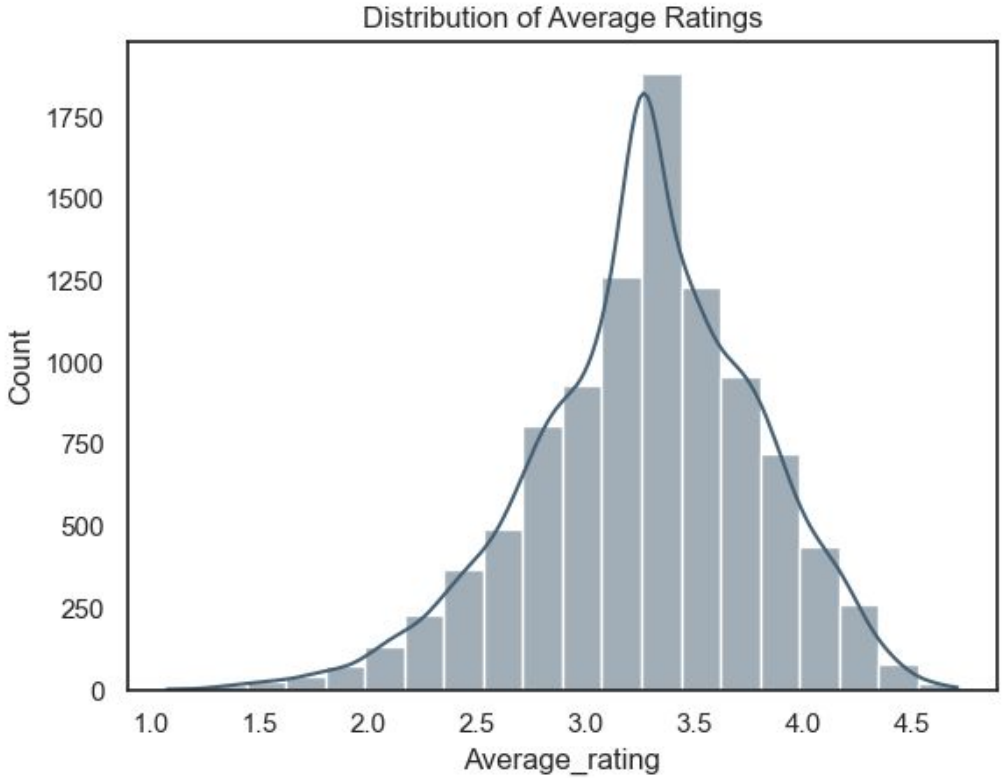
Attribute	Data Type
Watches	int64
List_appearances	int64
Li	int64
Fa	int64
Total_ratings	int64
Lo t★	int64
Me m★★★	int64
Hi st★★★★★	int64

1. Data after preprocessing

Average_rating	Runtime	Original_language	Watches	List_appearances	Total_ratings	Action	Comedy	...	Steven Spielberg	Tim Burton
3.79	102.0	English	1596190	254180	1076949	0	1	...	False	False

- 10000 entries
- 124 columns
- 19 genre columns
- 50 movie studio columns
- 50 director columns

2. Data exploration- rating

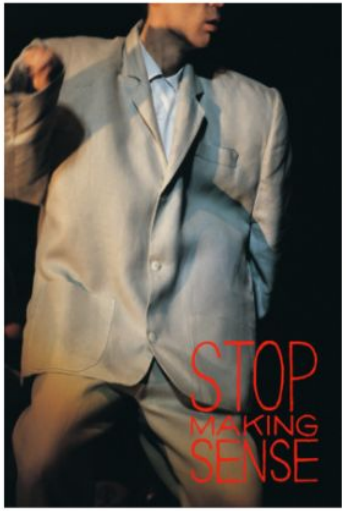


Letterboxd data is not perfect...

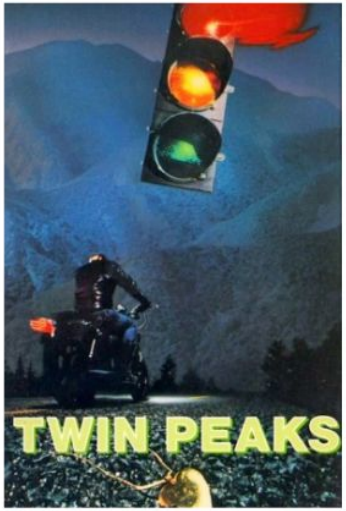
TOP 1

TOP 2

TOP 3

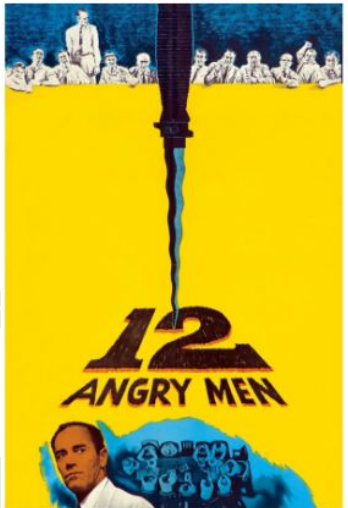


TOP 4

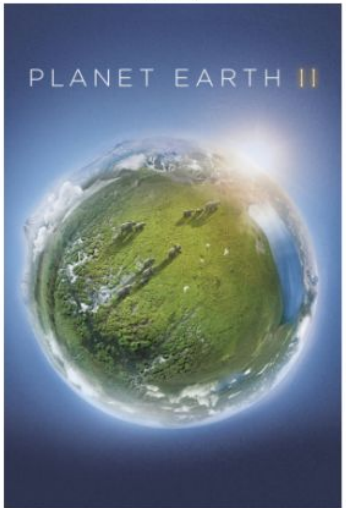


TOP 5

TOP 6

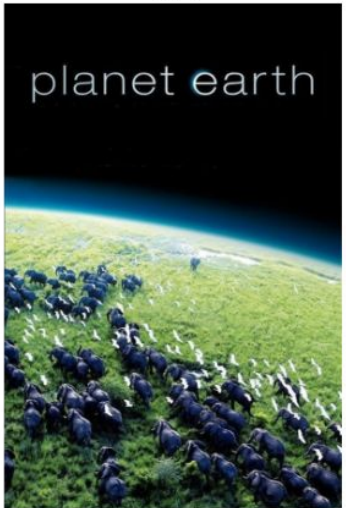


TOP 7

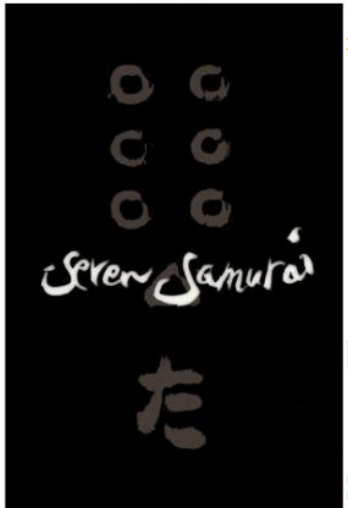


TOP 8

TOP 9

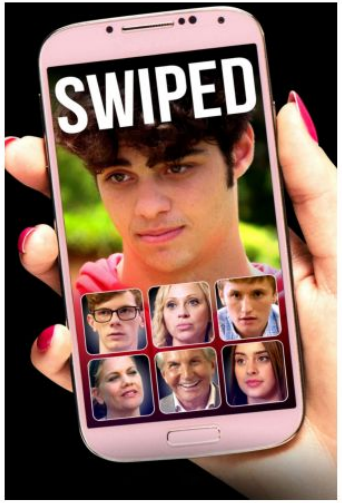


TOP 10



WORST 1

WORST 2



WORST 3

WORST 4

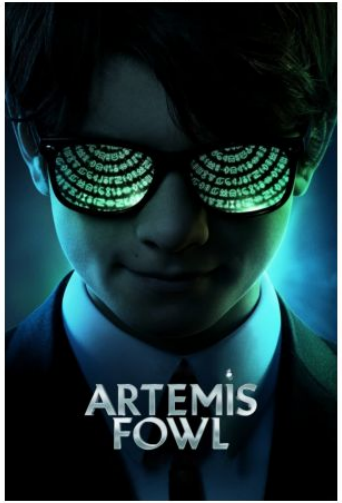
WORST 5

WORST 6

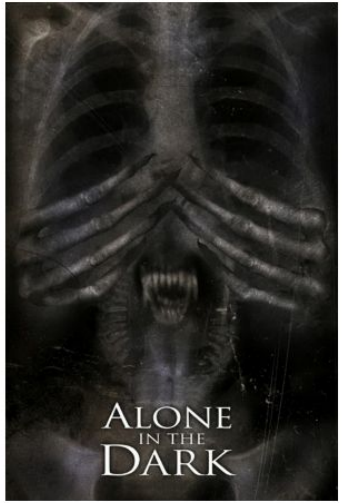
WORST 7

WORST 8

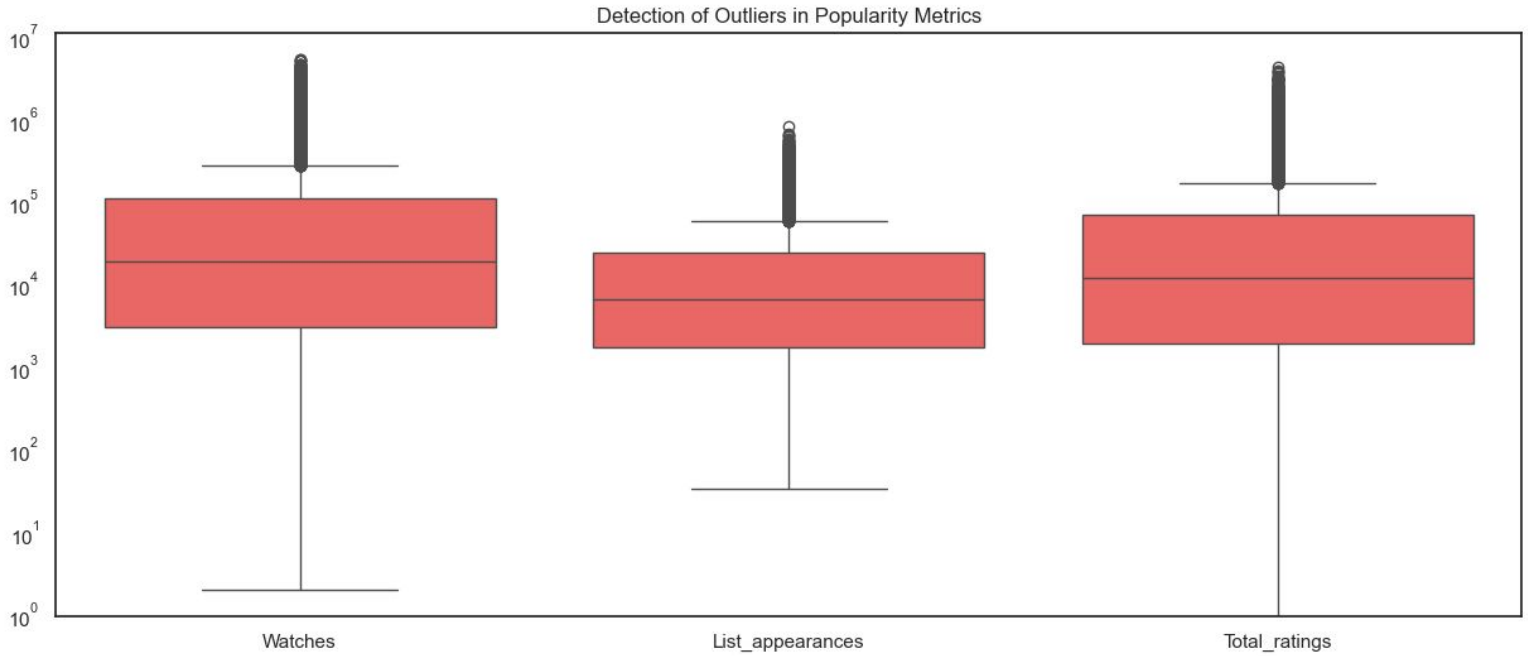
WORST 9



WORST 10

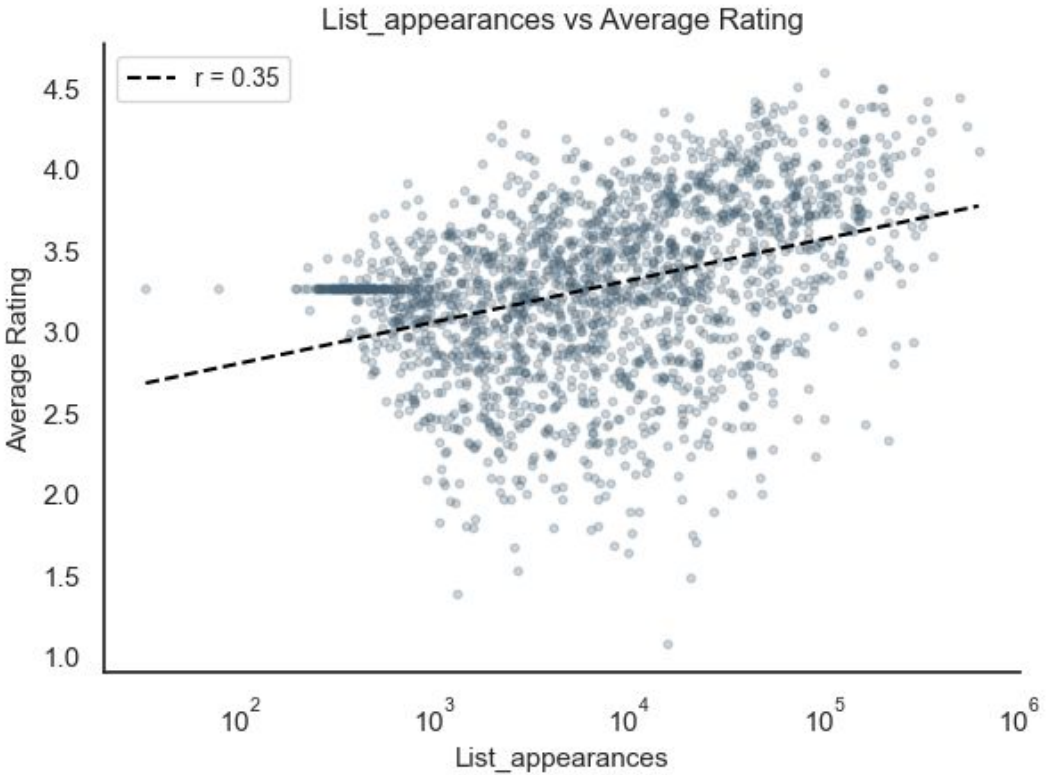


2. Data exploration- watches, lists, ratings

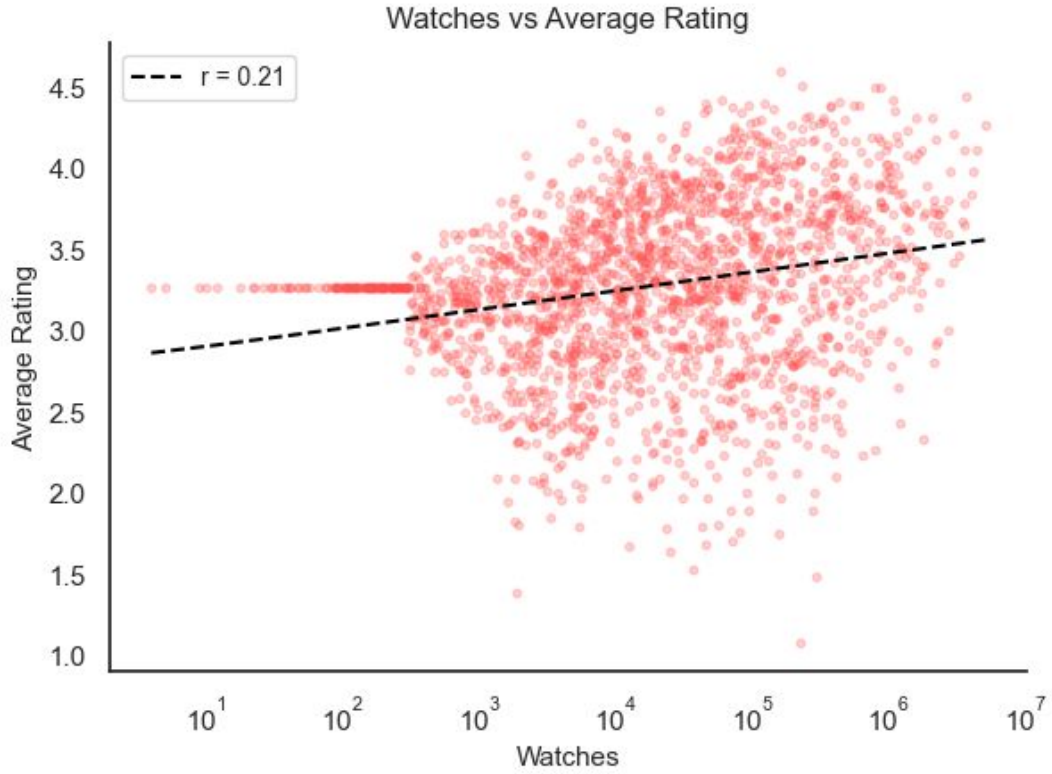


Despite the logarithmic scale, all metrics show extreme outliers

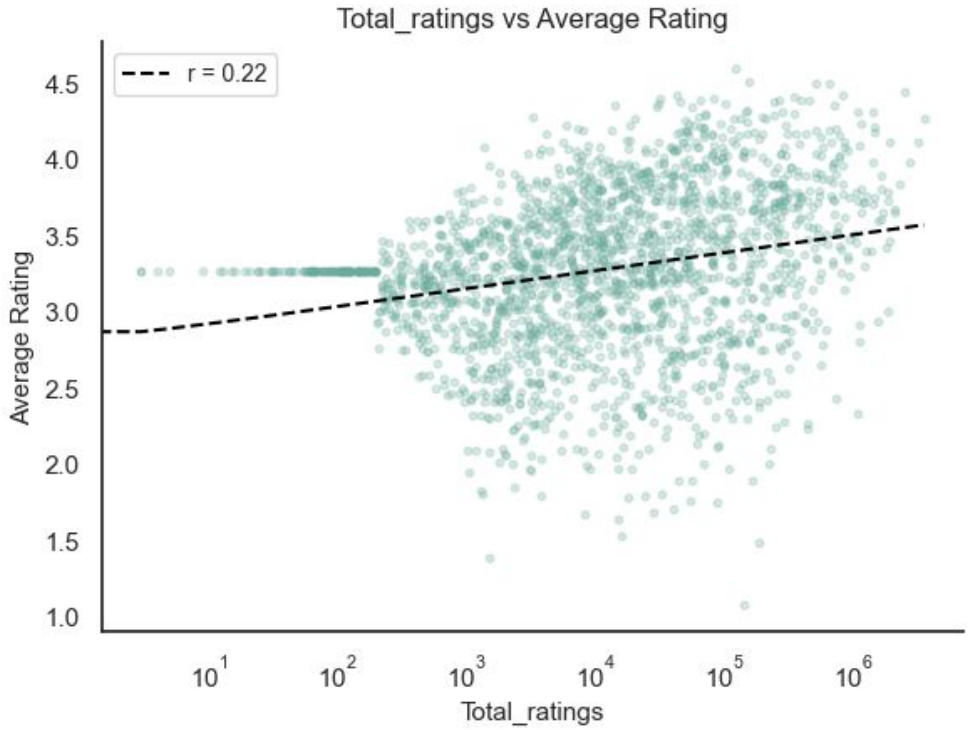
2. Data exploration- watches, lists, ratings

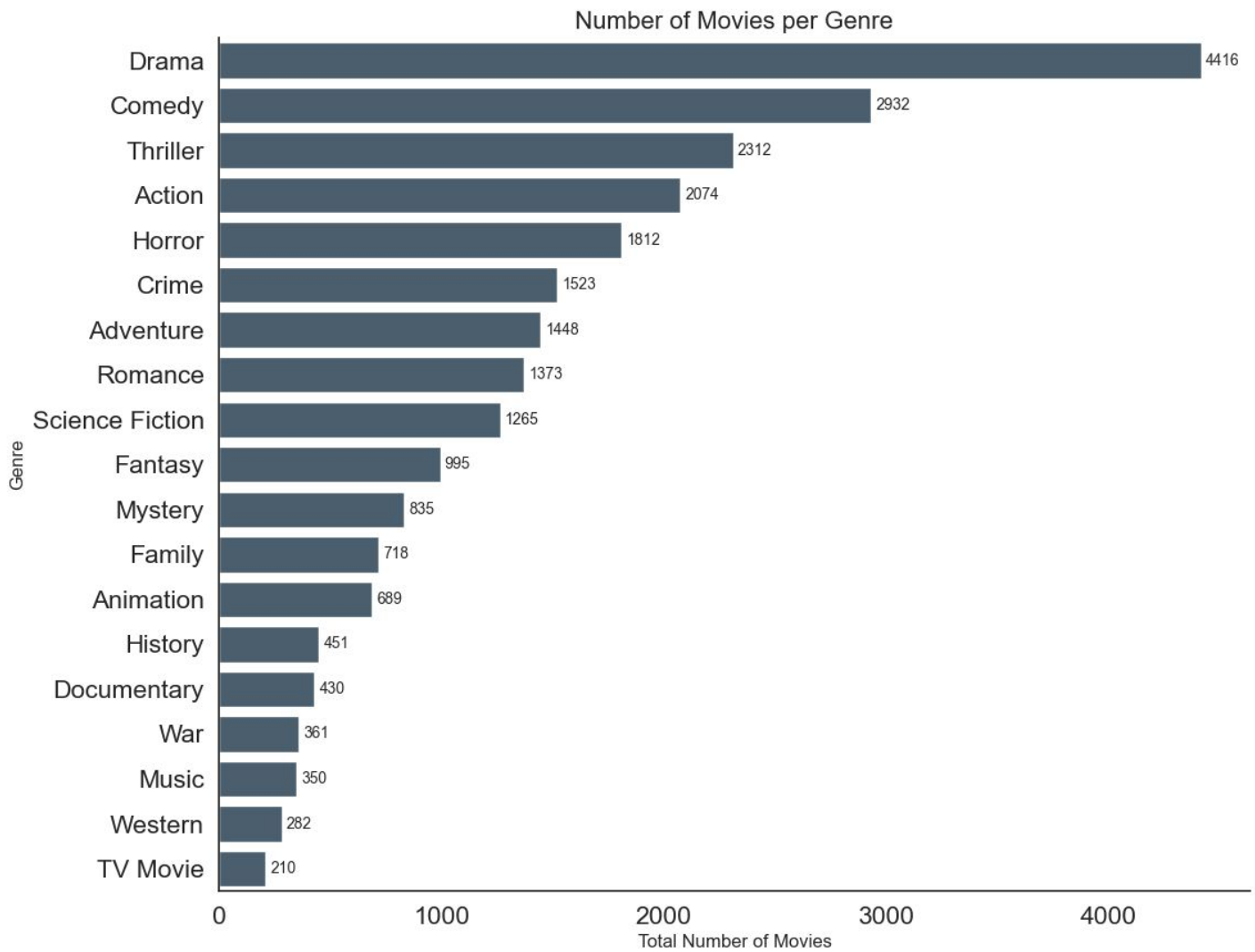


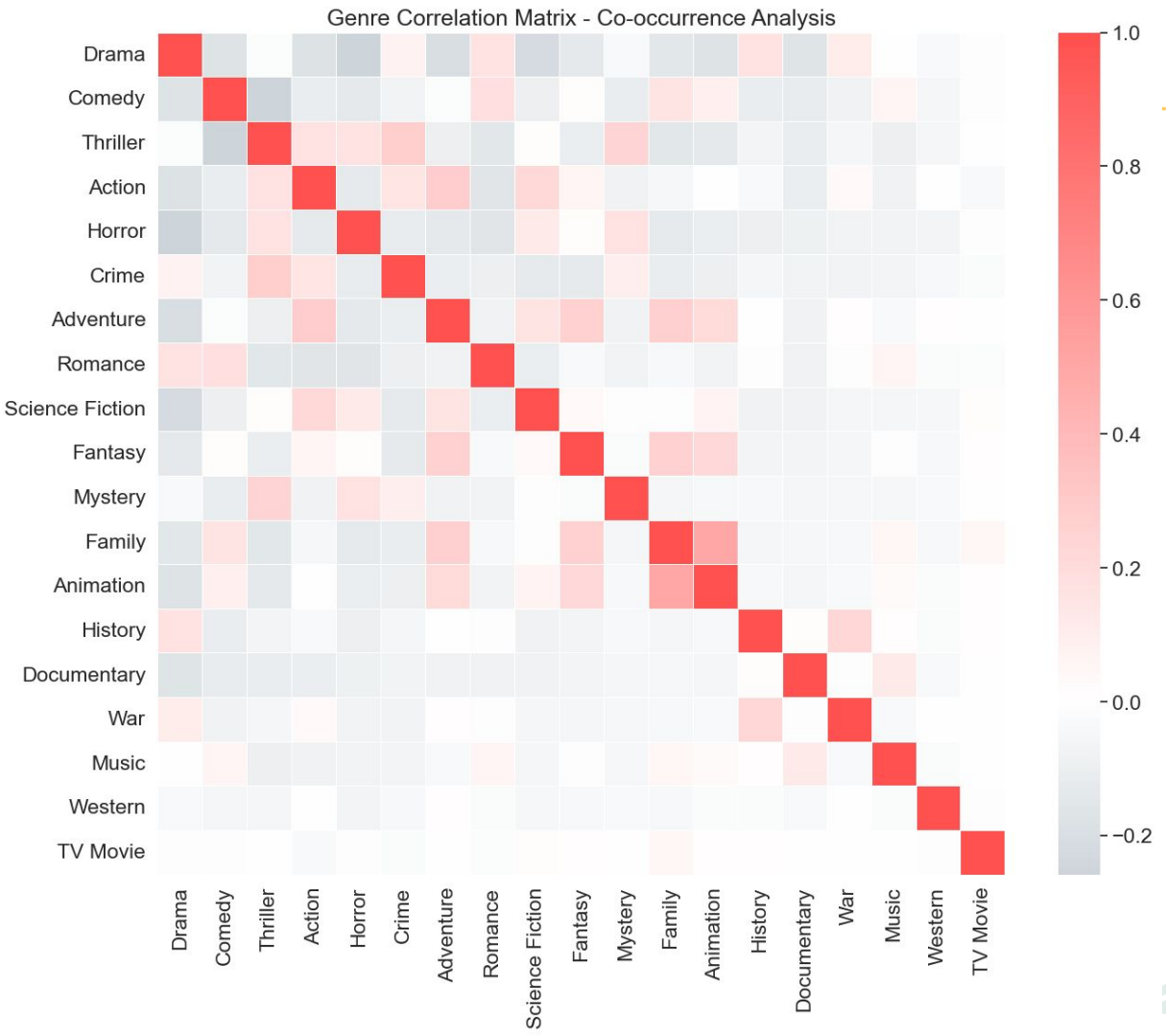
2. Data exploration- watches, lists, ratings

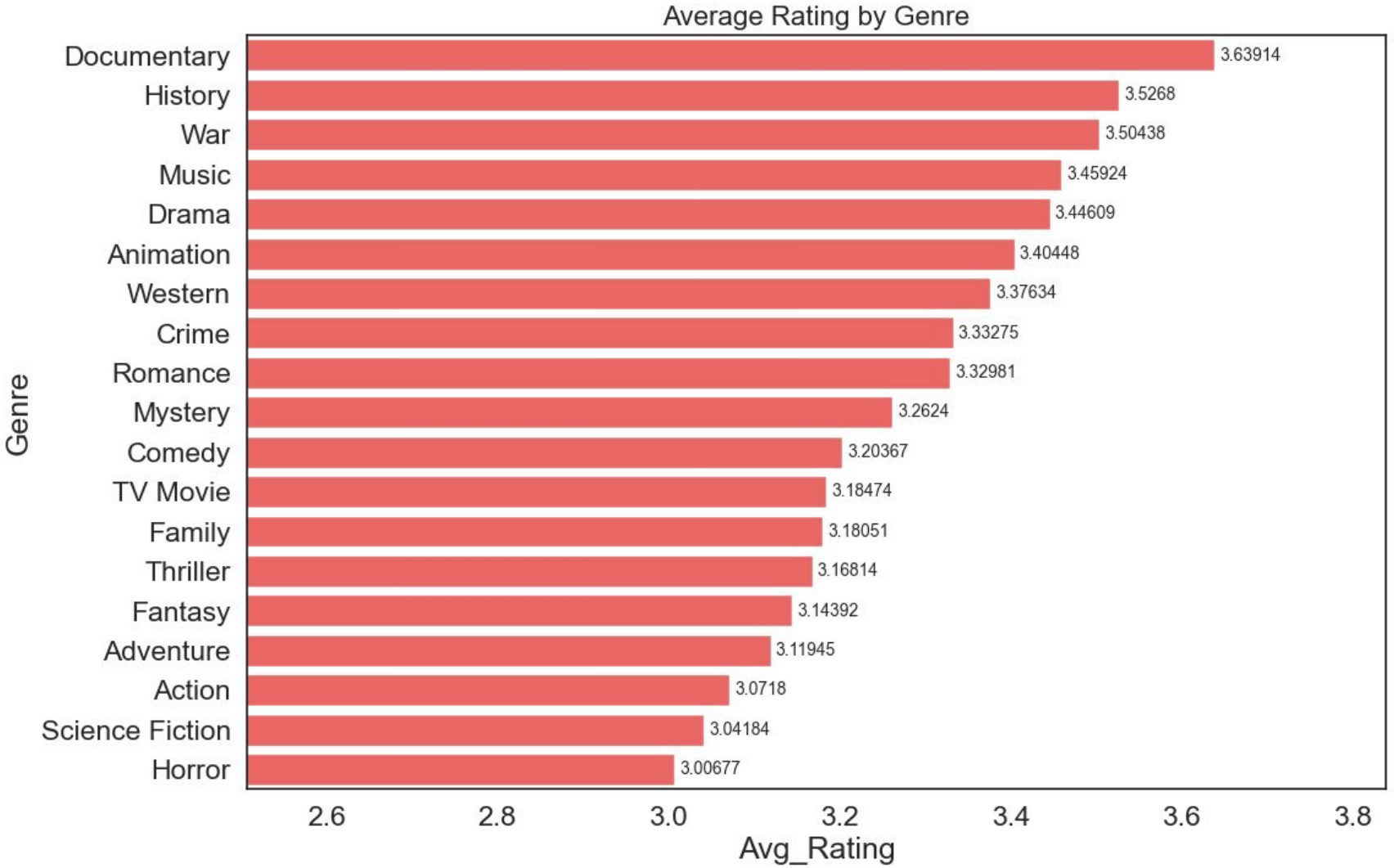


2. Data exploration- watches, lists, ratings

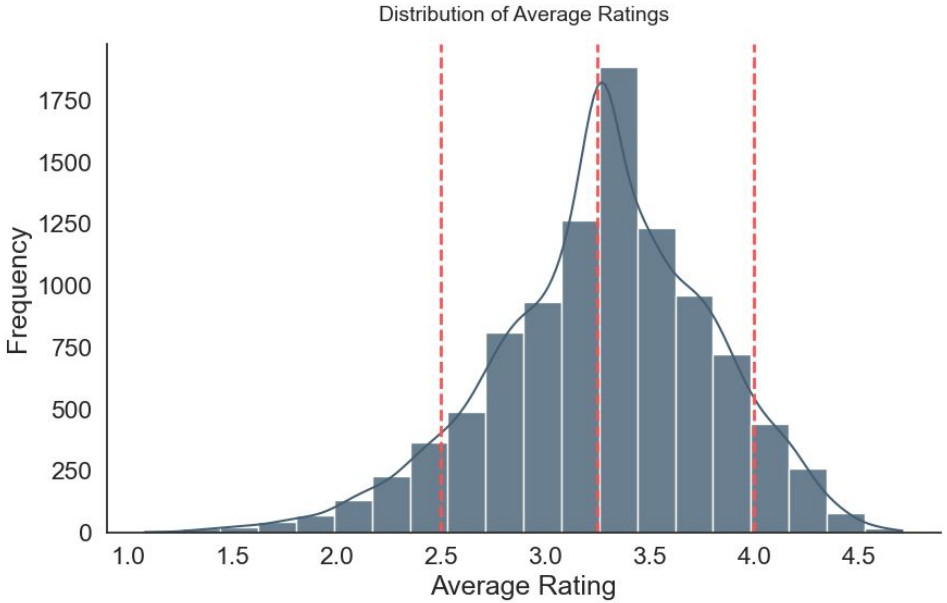


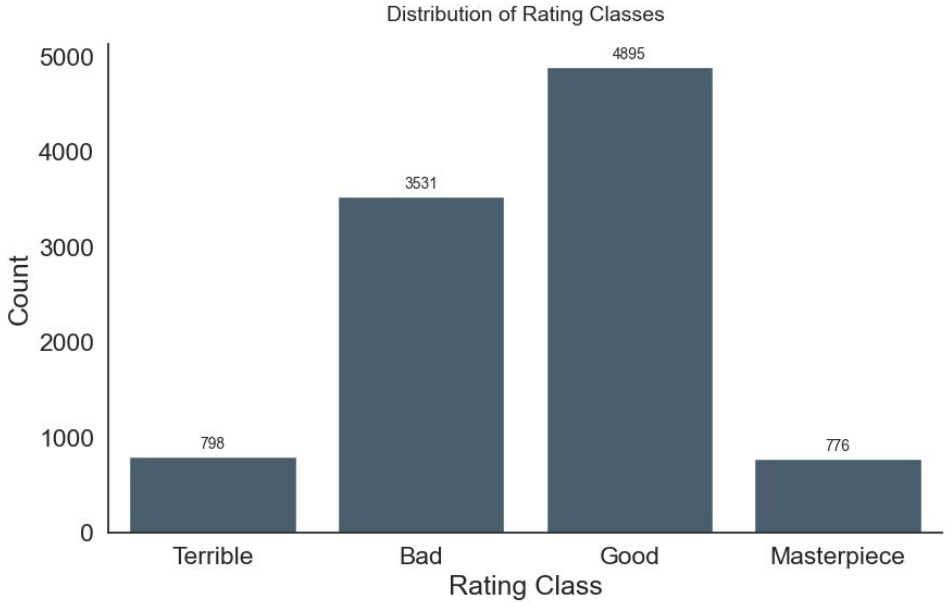

Number of Movies per Genre
Drama
Comedy
Thriller
Action
Horror
Crime
Adventure
Romance
Science Fiction
Fantasy
Mystery
Family
Animation
History
Documentary
War
Music
Western
TV Movie
4416
2932
2312
2074
1812
1523
1448
1373
1265
995
835
718
689
451
430
361
350
282
210
Genre
0
1000
2000
3000
4000
Total Number of Movies


Genre Correlation Matrix - Co-occurrence Analysis
Drama
Comedy
Thriller
Action
Horror
Crime
Adventure
Romance
Science Fiction
Fantasy
Mystery
Family
Animation
History
Documentary
War
Music
Western
TV Movie
1.0
0.8
0.6
0.4
0.2
0.0
−0.2


Average Rating by Genre
Documentary
History
War
Music
Drama
Animation
Western
Crime
Romance
Mystery
Comedy
TV Movie
Family
Thriller
Fantasy
Adventure
Action
Science Fiction
Horror
3.63914
3.5268
3.50438
3.45924
3.44609
3.40448
3.37634
3.33275
3.32981
3.2624
3.20367
3.18474
3.18051
3.16814
3.14392
3.11945
3.0718
3.04184
3.00677
Genre
2.6
2.8
3.0
3.2
3.4
3.6
3.8
Avg_Rating

2. Classification column one

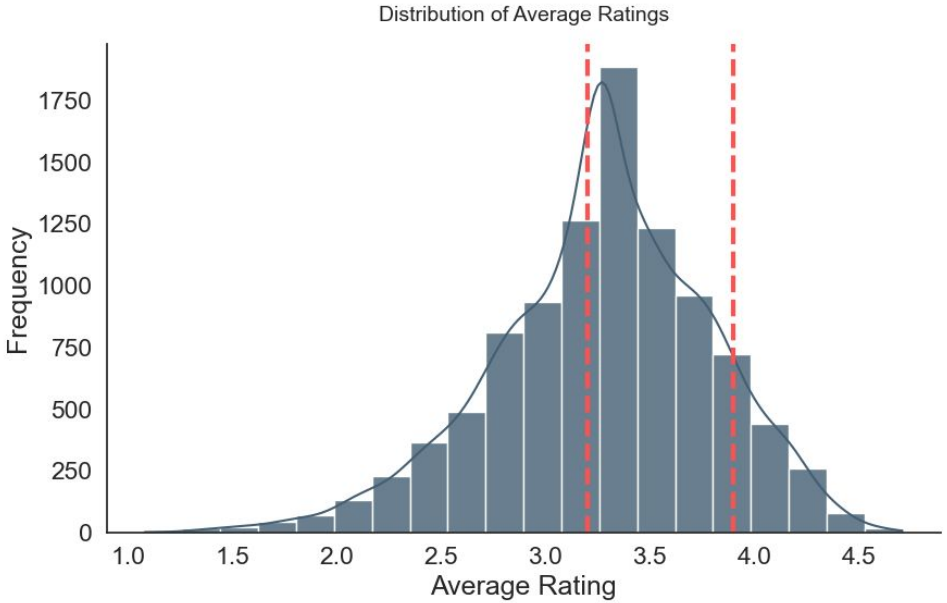


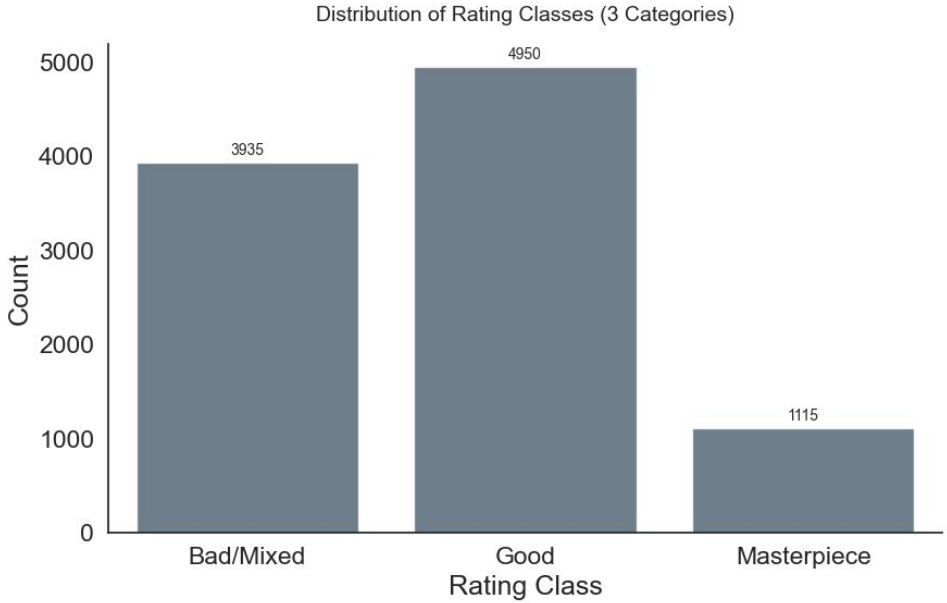


2. Classification column one

```
Rating_Class1 — baseline: always predict 'Good'
              precision    recall  f1-score   support

         Bad       0.00      0.00      0.00       698
        Good       0.49      1.00      0.66       989
  Masterpiece      0.00      0.00      0.00       137
     Terrible      0.00      0.00      0.00       176

    accuracy                           0.49      2000
   macro avg       0.12      0.25      0.17      2000
weighted avg       0.24      0.49      0.33      2000

Baseline Accuracy for Rating_Class1: 0.4945
```

2. Classification column two





2. Classification column one

Rating_Class2 — baseline: always predict 'Good'

	precision	recall	f1-score	support
Bad/Mixed	0.00	0.00	0.00	805
Good	0.49	1.00	0.66	986
Masterpiece	0.00	0.00	0.00	209
accuracy			0.49	2000
macro avg	0.16	0.33	0.22	2000
weighted avg	0.24	0.49	0.33	2000

Baseline Accuracy for Rating_Class2: 0.4930

3. Random Forest Classifier

Software & Tools

- Implemented in Python using Pandas for data tracking and scikit-learn / imbalanced-learn for machine learning modeling

Applied Classifiers

- Standard Random Forest (RFC)
- Balanced Random Forest

Model Parameters (Consistent across all models)

- n_estimators=500: Builds 500 decision trees per model
- max_features=0.5: Restricts each tree to evaluate only 50% of the available features at each split, encouraging model diversity and preventing overfitting
- random_state=67: Locks the random seed to guarantee that the experiment is 100% reproducible.

3. Random Forest results

RANDOM FOREST RESULTS

Classification Report for DF5 Rating_Class1:

	precision	recall	f1-score	support
Bad	0.64	0.74	0.68	698
Good	0.77	0.80	0.78	989
Masterpiece	0.64	0.53	0.58	137
Terrible	0.46	0.15	0.22	176
accuracy			0.70	2000
macro avg	0.63	0.55	0.57	2000
weighted avg	0.69	0.70	0.69	2000

Random Forest Test Accuracy for Rating_Class1: 0.7005

3. Random Forest results

BALANCED RANDOM FOREST RESULTS

Classification Report for DF5 Rating_Class1:

	precision	recall	f1-score	support
Bad	0.60	0.58	0.59	698
Good	0.83	0.65	0.73	989
Masterpiece	0.47	0.82	0.60	137
Terrible	0.30	0.53	0.38	176
accuracy			0.63	2000
macro avg	0.55	0.65	0.57	2000
weighted avg	0.68	0.63	0.64	2000

Random Forest Test Accuracy for Rating_Class1: 0.6260

3. Random Forest results

RANDOM FOREST RESULTS

Classification Report for DF5 Rating_Class2:

	precision	recall	f1-score	support
Bad/Mixed	0.79	0.79	0.79	805
Good	0.75	0.80	0.77	986
Masterpiece	0.74	0.57	0.64	209
accuracy			0.77	2000
macro avg	0.76	0.72	0.74	2000
weighted avg	0.77	0.77	0.77	2000

Random Forest Test Accuracy for Rating_Class2: 0.7680

3. Random Forest results

RANDOM FOREST RESULTS

Classification Report for DF5 Rating_Class2:

	precision	recall	f1-score	support
Bad/Mixed	0.76	0.82	0.79	805
Good	0.79	0.66	0.72	986
Masterpiece	0.55	0.79	0.65	209
accuracy			0.74	2000
macro avg	0.70	0.76	0.72	2000
weighted avg	0.75	0.74	0.74	2000

Random Forest Test Accuracy for Rating_Class2: 0.7400

3. Classification column one

Classifier Model	Target Variable	Accuracy	Weighted Precision	Weighted Recall	Weighted F1-Score
Standard Random Forest	Rating_Class1	0.70	0.69	0.70	0.69
Balanced Random Forest	Rating_Class1	0.63	0.68	0.63	0.64
Standard Random Forest	Rating_Class2	0.77	0.77	0.77	0.77
Balanced Random Forest	Rating_Class2	0.74	0.75	0.74	0.74

3. Random Forest- Top 10 features

Top 10 features — Random Forest — Rating_Class1:

	Importance
List_appearances	0.221555
Total_ratings	0.153603
Watches	0.142078
Runtime	0.114055
Original_language_English	0.034340
Drama	0.027730
Horror	0.027616
Documentary	0.021972
Action	0.016411
Science Fiction	0.014019

3. What if we delete some attributes?

Dataframe	Attributes	RFC rating_class1	RFC rating_class2	Balanced RFC rating_class1	Balanced RFC rating_class2
1	Runtime, Watches, List_appearances, Total_ratings	0.5950	0.6455	0.5235	0.6330
2	Runtime, Watches, List_appearances, Total_ratings, Original_language	0.6130	0.6720	0.5440	0.6460
3	Runtime, Watches, List_appearances, Total_ratings, Original_language, Genres	0.7015	0.7595	0.6115	0.7335
4	Runtime, Watches, List_appearances, Total_ratings, Original_language, Genres, Studios	0.6995	0.7610	0.6220	0.7340
5	Runtime, Watches, List_appearances, Total_ratings, Original_language, Genres, Studios, Directors	0.7005	0.7680	0.6260	0.7400

4. Balance for Boost Classifiers

To compare apples to apples (Boost Classifiers with BRF)
we use weights - smaller classes have higher weight

```
compute_sample_weight(class_weight='balanced', y=y1_train)
```

$$\text{Weight} = \frac{\text{n\_samples}}{\text{n\_classes} \times \text{count\_of\_class}}$$

4. (Balanced) XGBoost Classifier

Model Parameters

- **n_estimators**=500:
  Builds 500 decision trees per model
- **learning_rate**=0.05:
  After each tree is built, its predictions are multiplied by 0.05 before being added to the overall model
- **max_depth**=6:
  The maximum depth of any individual decision tree.
- **random_state**=67:
  Locks the random seed to guarantee that the experiment is 100% reproducible.
- **eval_metrics**='mlogloss':
  "Multiclass Logarithmic Loss" - measures how confident the model is in predicting the correct class, applying an exponentially harsher penalty the lower that correct probability is.

```
xgb_model = XGBClassifier(
    n_estimators=500,
    learning_rate=0.05,
    max_depth=6,
    random_state=67,
    eval_metric='mlogloss'
)
```

$$\text{mlogloss} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{j=1}^{M} y_{ij}\log(p_{ij})$$

4. Balanced XGBoost results

```
Classification Report for Rating_Class1:
              precision    recall  f1-score   support

           0       0.62      0.60      0.61       698
           1       0.82      0.71      0.76       989
           2       0.55      0.76      0.64       137
           3       0.34      0.56      0.42       176

    accuracy                           0.66      2000
   macro avg       0.58      0.65      0.61      2000
weighted avg       0.69      0.66      0.67      2000

XGBoost Test Accuracy for Rating_Class1: 0.6580
```

4. Balanced XGBoost results

```
Classification Report for Rating_Class2:
              precision    recall  f1-score   support

           0       0.76      0.84      0.80       805
           1       0.79      0.68      0.73       986
           2       0.57      0.76      0.65       209

    accuracy                           0.75      2000
   macro avg       0.71      0.76      0.73      2000
weighted avg       0.76      0.75      0.75      2000

XGBoost Test Accuracy for Rating_Class2: 0.7495
```

5. (Balanced) CatBoost Classifier

Model Parameters

- **iterations**=500:
  Builds 500 decision trees per model
- **learning_rate**=0.05:
  After each tree is built, its predictions are multiplied by 0.05 before being added to the overall model
- **depth**=6:
  The maximum depth of any individual decision tree.
- **random_state**=67:
  Locks the random seed to guarantee that the experiment is 100% reproducible.
- **loss_function**='MultiClass':
  Mathematically the same as in XGBoost

```
cat_model = CatBoostClassifier(
    iterations=500,
    learning_rate=0.05,
    depth=6,
    loss_function='MultiClass',
    random_seed=67,
    verbose=0
)
```

$$\text{MultiClass Loss} = -\frac{1}{N}\sum_{i=1}^{N}\log\left(\frac{e^{a_{it_i}}}{\sum_{j=0}^{M-1}e^{a_{ij}}}\right)$$

5. Balanced CatBoost results

Classification Report for Rating_Class1:

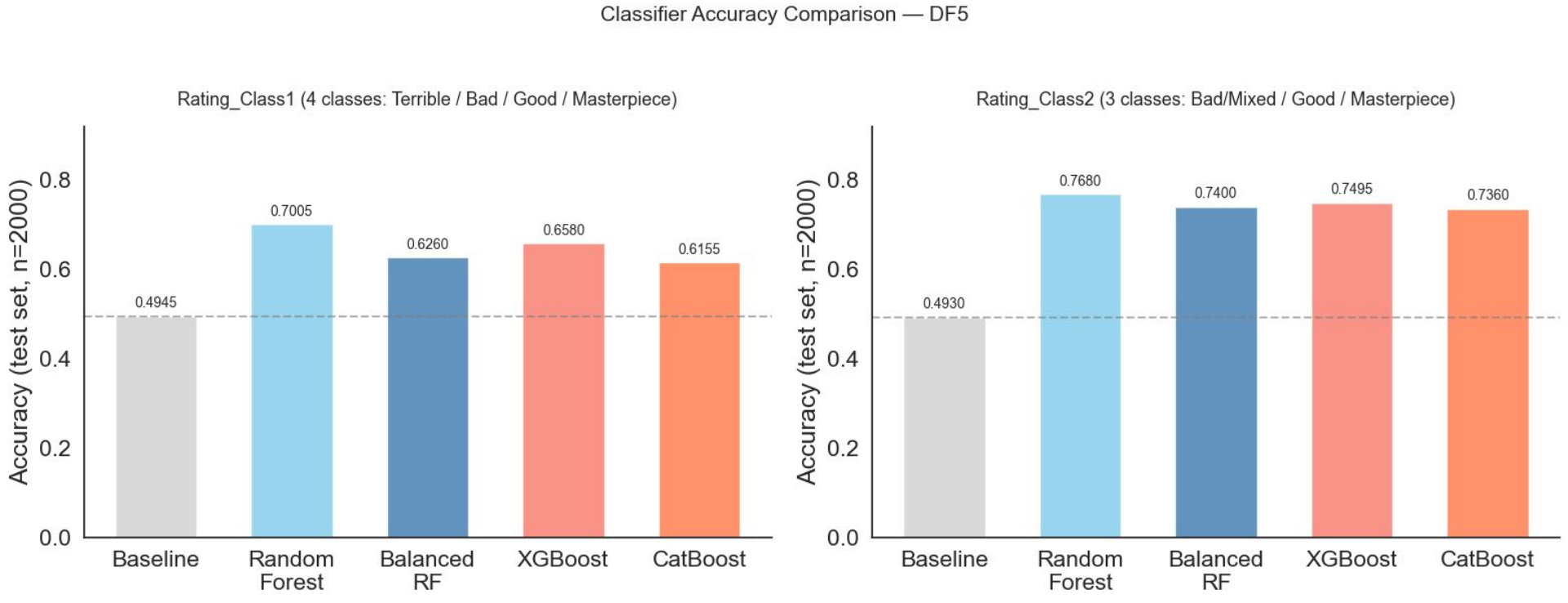
	precision	recall	f1-score	support
0	0.60	0.52	0.56	698
1	0.83	0.63	0.72	989
2	0.46	0.89	0.61	137
3	0.31	0.66	0.42	176
accuracy			0.62	2000
macro avg	0.55	0.68	0.58	2000
weighted avg	0.68	0.62	0.63	2000

CatBoost Test Accuracy for Rating_Class1: 0.6155

5. Balanced XGBoost results

```
Classification Report for Rating_Class2:
              precision    recall  f1-score   support

           0       0.75      0.83      0.79       805
           1       0.79      0.64      0.71       986
           2       0.54      0.82      0.66       209

   macro avg       0.70      0.76      0.72      2000
weighted avg       0.75      0.74      0.74      2000

CatBoost Test Accuracy for Rating_Class2: 0.7360
```

6. “Rating class 1” < “Rating class 2”



6. “Rating class 1” < “Rating class 2”

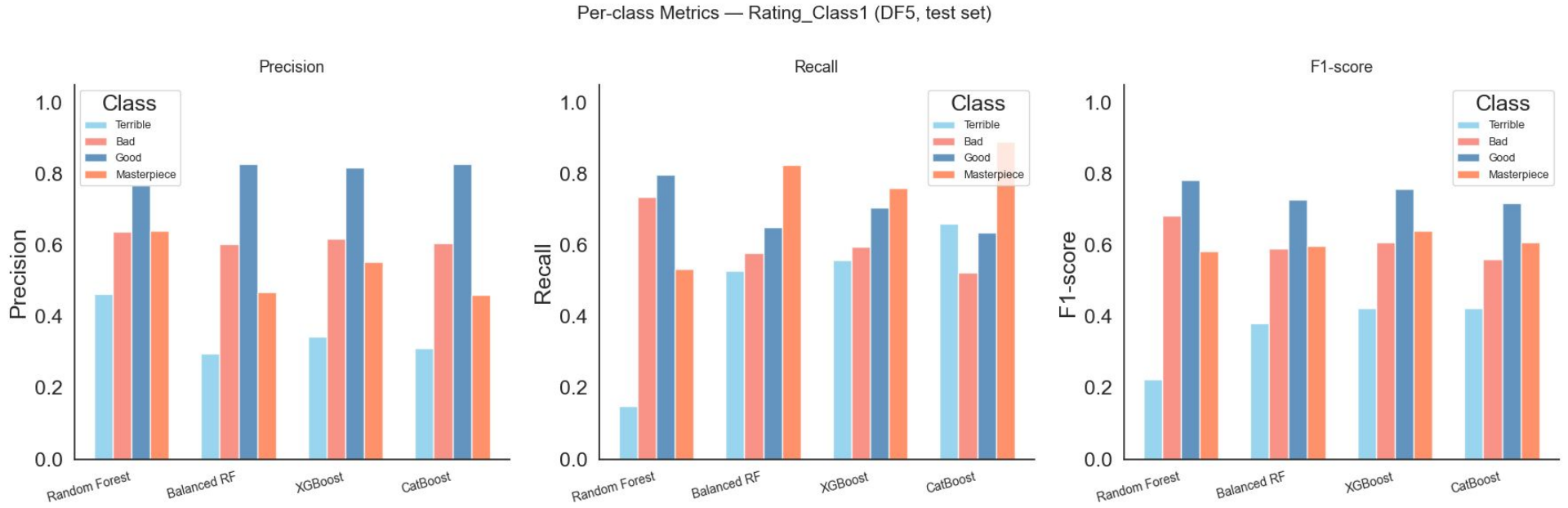
- Rating_Class2 consistently outperforms Rating_Class1 across all classifiers:

4 classes create more ambiguous boundaries — films near the cut-off between "Terrible" and "Bad", or between "Bad" and "Good", are genuinely hard to separate
Merging the two lowest categories into a single "Bad/Mixed" bucket removes the hardest boundary and results in a cleaner, more learnable problem.

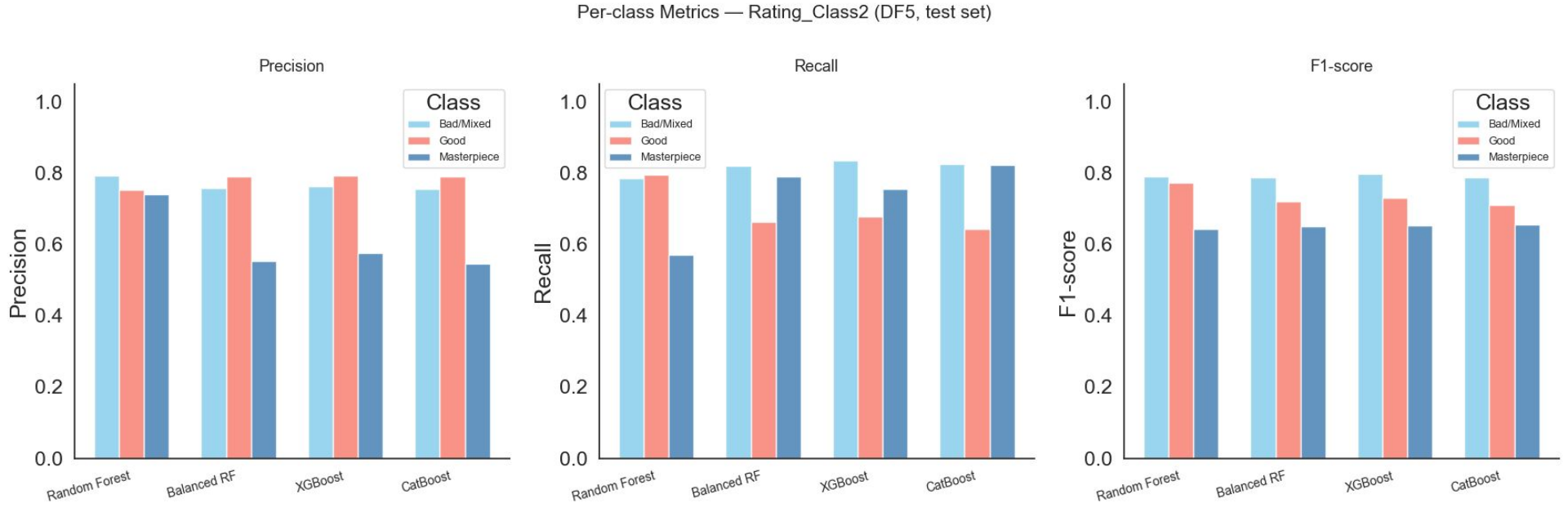
- Choosing the right target definition is at least as important as choosing the right classifier:

The biggest accuracy gains in this project come not from switching models, but from simplifying the classification task itself.

6. Precision vs Recall - tradeoff



6. Precision vs Recall - tradeoff



6. Precision vs Recall - tradeoff

- If we consider a “hard to guess” class "Masterpiece" we learn that:

  Standard Random Forest → higher precision and lower recall

  Balanced Classifiers → lower precision and higher recall

- The standard RF is the best choice if we care about **confidence** when predicting a Masterpiece – when it says "Masterpiece", it is right 74% of the time.

- The balanced models trade that confidence for **coverage** – they catch far more actual Masterpieces, but also produce more false positives.

Thank you